09 · Documento de Cobertura del Testing · EP2

Documento editable para completar con la evidencia real del proyecto

1. Identificación

Campo	Completar
Sección	001-D
Grupo	Grupo 1
Integrantes	Ignacia Padilla y Elba Sanchez
Contexto / caso	Forma B — Sonido Vivo
Repositorio GitHub	https://github.com/NovaFugaz/super-enigma
Versión evaluada	

2. Configuración del entorno de pruebas

Describa las herramientas y versiones utilizadas, los scripts definidos en package.json y cualquier configuración relevante.

Framework de pruebas: Vitest
Biblioteca de componentes: React Testing Library
Otros paquetes relevantes: jsdom, @testing-library/jest-dom, @vitejs/plugin-react
Comando de ejecución: vite.config.js define environment: 'jsdom' y carga setup.js
Comando de cobertura: Aún no configurados los tests de cobertura. Se ha privilegiado en esta etapa del proyecto de generar tests para probar la lógica funcional con datos demo y persistencia simulada por localStorage, no con componentes adicionales.

3. Registro de pruebas

Nº	Archivo / componente	Comportamiento verificado	Tipo	Resultado
1	adminData.test.jsx	Gestión de usuarios y roles del panel admin; creación, edición y desactivación	Integración	OK
2	catalogWorkbook.test.js	Parseo de planillas, validación de columnas, precios y stock inválidos, códigos duplicados	Unitario	OK
3	mockAuth.test.js	Seed de cuentas demo, autenticación, registro, cambio de contraseña y cuentas inactivas	Unitario/Integración	OK
4	mockOrders.test.js	Semillas de pedidos, pago, stock, transiciones de estado y búsqueda por tracking	Unitario/Integración	OK
5	ProtectedRoute.test.jsx	Acceso protegido por roles y redirecciones según autenticación	Componente	OK
6	salesReport.test.js	Generación de CSV, escape de comillas y reporte vacío	Unitario	OK
7	cartAndForms.test.jsx	Validación de carrito, cálculo de totales y formularios de login/registro	Componente/Unitario	OK
8	secondaryForms.test.jsx	Recuperación de contraseña y selección de sucursal de contacto	Componente	OK
9	cartCheckout.test.jsx	Revisión de requisitos de cliente, compra aprobada y pago rechazado	Integración	OK
10	checkoutFlow.test.jsx	Agregado al carrito desde detalle, pedido confirmado y seguimiento por tracking	Integración	OK

Resultado: 10 archivos de prueba pasados | 49 pruebas pasadas

4. Mocks y spies utilizados

Se utilizaron principalmente servicios simulados y almacenamiento local para validar reglas de negocio sin depender de un backend real. Como se mencionó anteriormente, lo que se testeó es mayoritariamente funcionalidad y comportamiento real, usando localStorage y datos de demostración. Lo principal que se evaluó:

- mockAuth.js: simula autenticación, registro y gestión de usuarios.
- mockOrders.js: simula pagos, pedidos, stock y estados del flujo.
- CatalogContext.jsx: almacena y actualiza stock y productos en localStorage.
- cart.js: encapsula lógica del carrito y cálculo de totales.

5. Resultado de cobertura

Métrica	Resultado	Interpretación
Statements	N/A %	No disponible / no configurado
Branches	N/A %	No disponible / no configurado
Functions	N/A %	No disponible / no configurado
Lines	N/A %	No disponible / no configurado

6. Análisis

- ¿Qué componentes o funciones quedaron mejor cubiertos?
  - Autenticación y gestión de usuarios
  - Gestión de catálogo y stock
  - Órdenes y flujo de pago
  - Rutas protegidas
  - Carrito y checkout
  - Seguimiento de pedido

- ¿Qué funcionalidades presentan menor cobertura y por qué?
  - Componentes más visuales sin lógica de negocio fuerte
  - Flujos de experiencia de usuario más complejos
  - Integración real con backend y persistencia externa

- ¿Qué error o comportamiento inesperado fue detectado gracias a una prueba?
  - Se detectó una referencia no definida en ProductDetailPage.jsx, al usar una variable notice que no existía en el componente.
  - La prueba de flujo de compra reveló ese problema y permitió corregirlo.

- ¿Qué prueba adicional sería conveniente implementar en una siguiente versión?
  - Cobertura con backend real cuando exista
  - Un caso de flujo completo E2E de compra desde catálogo hasta confirmación final

7. Conclusiones

En nuestro caso, desarrollar los tests, aporta, en términos técnicos, una mejora a la calidad del frontend, que es la etapa en la que estamos actualmente, validando comportamientos críticos, flujos y reglas de negocio. En SonidoVivo, automatizar las pruebas nos permitió detectar errores de validación, inconsistencias en la gestión de autenticación y fallas de renderizado en componentes clave, reforzando la estabilidad, mantenibilidad y confiabilidad del desarrollo. En términos prácticos, el testing evita regresiones, ayuda a mejorar y fortalecer el código y aumenta la confianza en cada cambio funcional realizado, en este caso, en la interfaz.